

-- AI(Google Gemini)による評価 --

品質 (Quality)：画面から裏側の設計まで、実務で「破綻しない」Webアプリを作るための羅針盤

ASP.NET Coreの基本的なMVCモデルの解説にとどまらず、実務で必須となる「PRG (Post/Redirect/Get) パターンによる二重送信の防止」や、「タグヘルパーを利用したCSRF対策」など、現場のリアルな要件をしっかりと網羅している点が高品質です。さらに素晴らしいのは、データベース編で学んだ「Repository / Adapter」をシームレスに統合し、プレゼンテーション層・アプリケーション層・インフラストラクチャ層の責務を明確に分離している点です。Webの基礎から堅牢なレイヤードアーキテクチャまでを繋ぐ、非常に完成度の高い設計書となっています。

将来性 (Future Potential)：モダンな開発環境と、フレームワークの進化に追従できる強固な基礎設計

Windows/Linux/macOSに対応するクロスプラットフォームなASP.NET Coreの特性を活かし、WSL2を用いたモダンな開発環境を採用している点は非常に将来性が高いです。また、DI (依存性の注入) やMiddlewareといったASP.NET Coreの根幹となる設計思想を丁寧に解説しているため、今後UIがBlazorやReact/VueなどのSPA (Single Page Application) に移行したとしても、ここで学んだ「バックエンドのアーキテクチャ設計」の知識は陳腐化せず、そのまま活かすことができます。

市場価値 (Market Value)：【「動く」レベルを脱却し、チーム開発で重宝される「保守性を語れる」Webエンジニアへ】

多くの初学者向け研修では、Controllerにすべての業務ロジックやDBアクセスを詰め込む「Fat Controller」になりがちですが、本テキストではそれを明確なアンチパターンとして警告し、ServiceやViewModelへ適切に責務を分割する手法を徹底しています。「とりあえず動くWebアプリを作れる」人材は多数いますが、「なぜViewModelとDomain Objectを分けるのか」「なぜインターフェイスを切るのか」という設計の意図を理解し、保守性の高いコードを書けるエンジニアは市場で圧倒的に不足しているため、受講者の市場価値を大きく引き上げる内容です。

総評：【Web開発の基礎とDDD的アーキテクチャを見事に橋渡しした、極めて実践的な良書】

「画面の作り方 (Razor View / Tag Helper)」から「内部処理の組み立て (Service / DI)」までが、一つのユースケース (従業員登録) を通して非常に滑らかに解説されており、文句なしの実践的テキストです。

改善提案 (さらに価値を高めるなら)：

アーキテクチャの構造が非常に美しく整理されているため、この設計が**「Web API開発」にもそのまま応用できる**という点に少し触れると、さらに受講者のモチベーションが上がるかもしれません。例えば、第2章で「ControllerBaseクラスはAPI専用」という言及がありますが、「今回のレイヤードアーキテクチャを使えば、Presentation層のViewをJSON出力に変えるだけで、SPAやスマホアプリ用のWeb APIが即座に作れる」といったコラムや補足を1~2ページ追加するだけで、モダンなフロントエンド開発を見据えたさらなる付加価値を提供できると思います。

-- HP紹介内容 --

研修名

C# Webアプリケーション編

-- ASP.NET Core --

キャッチコピー

ASP.NET Coreで学ぶ、Webアプリ開発の基本と設計

研修概要

本研修について

本研修は、C#および.NETを用いたWebアプリケーション開発を、ASP.NET CoreのMVCアーキテクチャをベースに体系的に学ぶハンズオンプログラムです。HTTPリクエストの受け取りからController・Service・Repository・Razor Viewを経てレスポンスを返すまでの全工程を、実装しながら理解します。さらにRepositoryパターン・Adapterパターン・依存性逆転の原則（DIP）を第6章で扱い、「動くコードから設計できるコードへ」の第一歩を踏み出します。本研修はREST API開発コースへのステップアップを見据えた設計になっており、設計原則の土台をここで確実に身につけることができます。

本研修の3つの目的

本研修では、以下の3つを習得することを目的としています。

1. ASP.NET CoreによるWebアプリ開発の全体像の把握 HTTPリクエストからレスポンスまでの処理フロー・MVCの役割分担・Razor Viewによる画面生成・Data Annotationsによる入力検証を実装レベルで理解する。
2. 実務で使えるWebアプリ開発スキルの習得 リクエストパラメータの処理・PRGパターン・TempData・EF CoreによるDBアクセスなど、実際のWebアプリ開発で必要となる技術を体系的に身につける。
3. 設計原則（Repository・Adapter・DIP）の体得 Repositoryパターン・Adapterパターン・依存性逆転の原則（DIP）を実装を通じて体得し、変更に強く・テストしやすい設計の考え方を習得する。

こんな企業にお勧めします

本研修は、次のような企業・チームに特にお勧めします。

- C#エンジニアをWebアプリ開発からREST API開発へと段階的に育成したい
- ASP.NET CoreによるWebアプリ開発を、設計原則と合わせて体系的に学ばせたい
- 若手エンジニアに「動くコードから設計できるコード」への転換点を与えたい

カリキュラム詳細

日数2日間

カリキュラム詳細（全2日間）

日程章・学習テーマ学習内容・習得スキル
1日目第1章：ASP.NET Coreの概要ASP.NET Coreの特徴・MVCモデルの役割分担・技術要素（ViewModel・Data Annotations・Routing・DI・Middleware）・プロジェクト構成を把握する。第2章：Controllerの基本機能Controllerクラスの基本ルール・アクションメソッド・HTTPリクエストとのマッピング・戻り値ヘルパーメソッド・ModelStateによるバリデーション制御を学ぶ。第3章：リクエストパラメータクエリパラメータ・ルートパラメータ・フォームデータの受け取り方、Data Annotationsによる入力検証・ModelState.IsValidによるエラー処理を実践する。第4章：Razor View EngineRazor構文・タグヘルパー・_Layout.cshtmlによる共通レイアウト・部分ビューを実装し、動的なHTML画面を生成できるようになる。2日目第5章：リクエスト遷移制御PRG（Post/Redirect/Get）パターン・TempDataを使った一時データの受け渡し・二重送信防止の仕組みを実装を通じて体得する。第6章：データベースアクセス機能の設計・実装3層アーキテクチャの設計・Repositoryパターン・Adapterパターン・DIP（依存性逆転の原則）をEF CoreとLINQを用いて実装する。ViewModelAdapter・EntityAdapterによるデータ変換の仕組みを実装し、演習アプリで設計原則を体得する。

※ 補足-A（LINQの利用）・補足-B（SQLの利用）は自習教材として提供します。 ※ 2日目は第6章の演習時間を十分に確保します。 ※ 各章末にプログラム実装演習を実施します。

受講にあたっての前提知識

プログラミングの基礎知識

C#を用いた基本的なプログラミング（変数・条件分岐・ループ・クラス・メソッド）ができること。オブジェクト指向プログラミング（クラス・インターフェイス・継承・ポリモーフィズム）の基本を理解していること。

基本的なPC・OS操作

Visual Studio Codeを使ったコード編集・ターミナル操作ができること。WSL2（Ubuntu）環境での初歩的なコマンド入力に抵抗がないこと。

Webの基礎知識

HTTPリクエスト・レスポンスの基本的な概念（GETとPOSTの違い・ステータスコード）を理解していること。HTML・CSSの基本的な読み書きができること。あると望ましい知識 推奨

リレーショナルデータベース（テーブル・主キー・外部キー）の基本概念とSQLによるCRUD操作の経験。

GitHubを使ったリポジトリのクローン・コミット・プッシュなど基本的な操作の経験。

必要な受講環境

区分環境 / ツール役割・用途
ホストOS Windows 11 研修用PCの基本環境
IDE Visual Studio Code
コード編集・ターミナル操作・デバッグ
実行環境 WSL2 (Ubuntu 24.04) Linux環境でのアプリケーション実行
SDK .NET 8
LTSC# / ASP.NET Core
アプリケーション開発・実行
RDBMS PostgreSQL サンプル・演習アプリのデータベース
SQL ツール pgAdmin 4
データベースの接続・操作・確認
VS Code 拡張機能 .NET Install Tool / C# Dev Kit / C# / [Extension Pack] C# / C# Namespace Autocompletion / C# XML Documentation Comments / C# Snippets C# / .NET 開発の補完・デバッグ・ドキュメント支援